KUBERNETES NOTES

— Page 1 —

1. What is Kubernetes ?

Kubernetes (K8s) is an open-source container orchestration platform.
It helps us to automate the deployment, scaling, management and networking of containers.
It was originally developed by Google and is now maintained by CNCF.

In Simple Words

Kubernetes helps us to run our containerized applications reliably at scale.

2. Why Kubernetes was Created ?

Earlier we were using Docker to run containers.
But managing multiple containers manually was very difficult.
Kubernetes solves these problems.

Problems with Docker Alone

- Hard to manage multiple containers
- No auto scaling
- No self-healing if container fails
- No load balancing
- No rolling updates
- Difficult to manage networking and storage

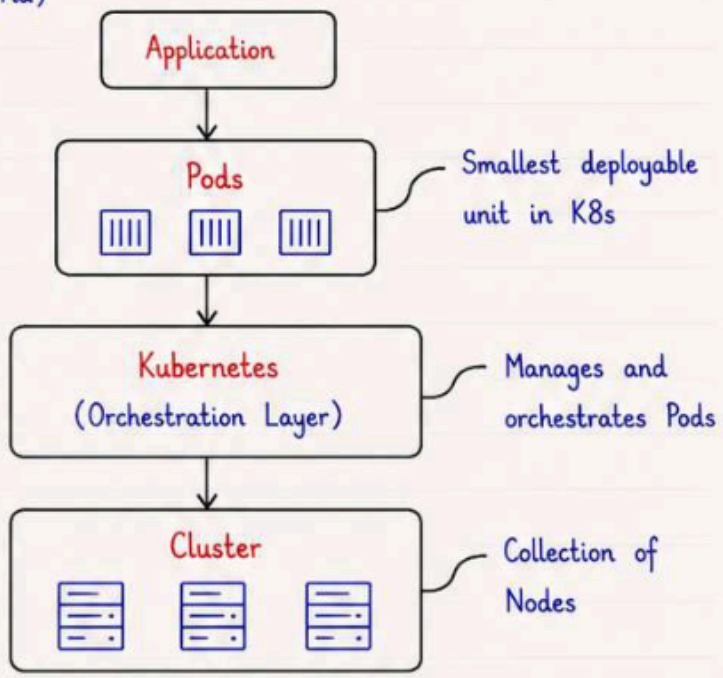
3. Features of Kubernetes

- Automatic deployment and scaling
- Self-healing (restarts failed containers)
- Load balancing and service discovery
- Rolling updates and rollback
- Storage orchestration
- Secret and configuration management
- Runs on any environment (On-prem, Cloud, Hybrid)

4. Container Orchestration

Container orchestration means managing the entire lifecycle of containers like deploying, scaling, networking, monitoring, etc. automatically.

Docker vs Kubernetes	
Docker	Kubernetes
Manages single container	Manages multiple containers
Manual process	Automatic process
No scaling, self-healing	Auto scaling, self-healing



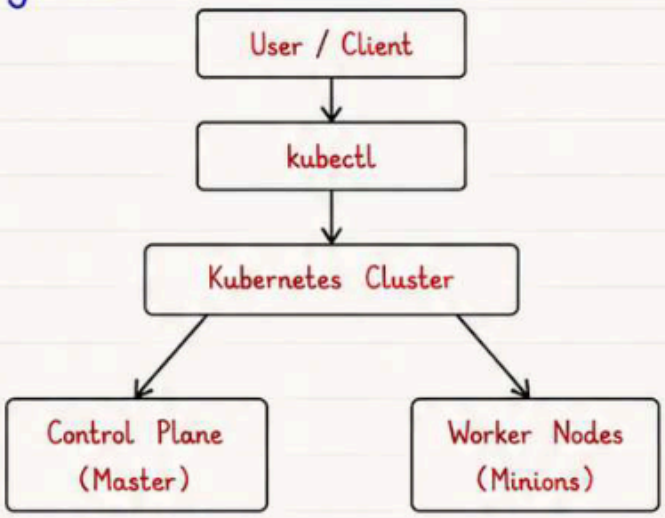
KUBERNETES NOTES

— Page 2 —

1. Kubernetes Architecture

Kubernetes follows a Master-Worker (Client-Server) architecture.
The Cluster is divided into two main parts :

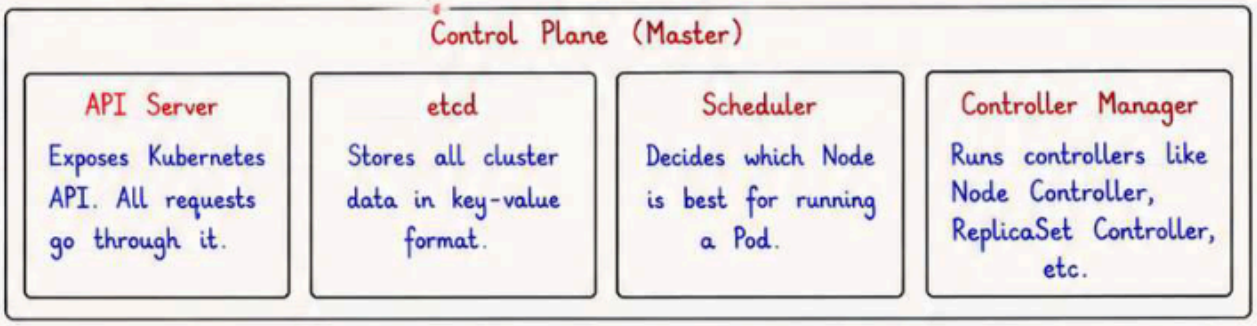
- Control Plane (Master)
- Worker Nodes



2. Control Plane Components

Control Plane makes global decisions about the cluster (e.g. scheduling), detects and responds to cluster events and maintains the desired state.

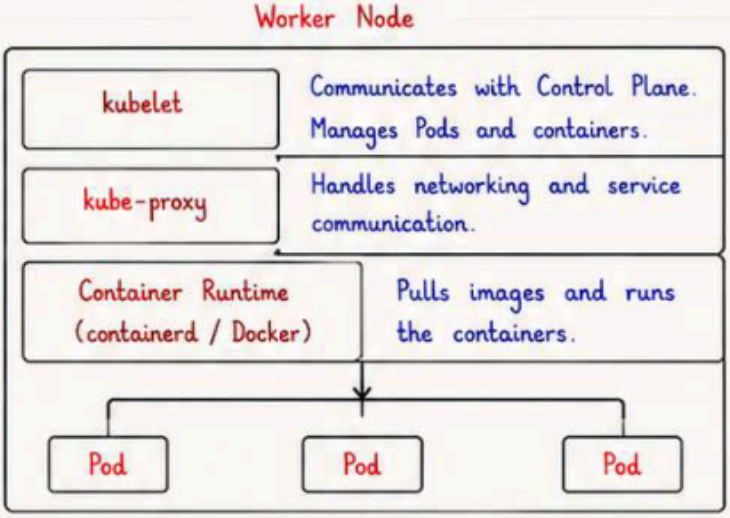
- API Server : Frontend of Kubernetes. Exposes the REST API used by kubectl and other components.
- etcd : Consistent and highly-available key value store for all cluster data.
- Scheduler : Watches for newly created Pods and selects a Node for them to run on.
- Controller Manager : Runs controller processes that regulate the state of the cluster.



3. Worker Node Components

Worker Nodes run the containerized applications and maintain the running environment.

- kubelet : Ensures containers are running in a Pod.
- kube-proxy : Maintains network rules for communication.
- Container Runtime : Runs the containers (e.g. containerd, Docker).



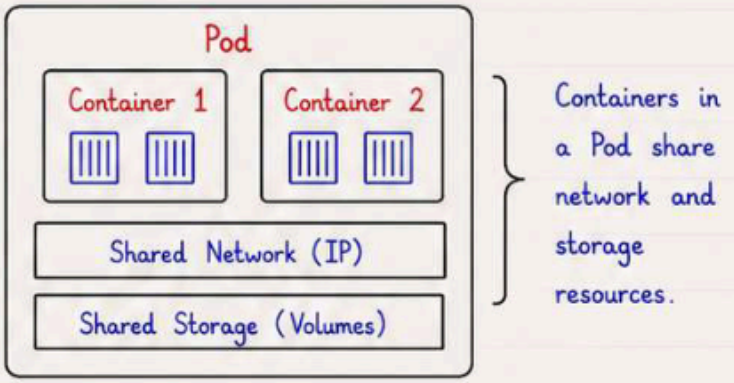
Control Plane manages the Cluster, while Worker Nodes run the applications. Together, they make Kubernetes powerful and reliable.

KUBERNETES NOTES

— Page 3 —

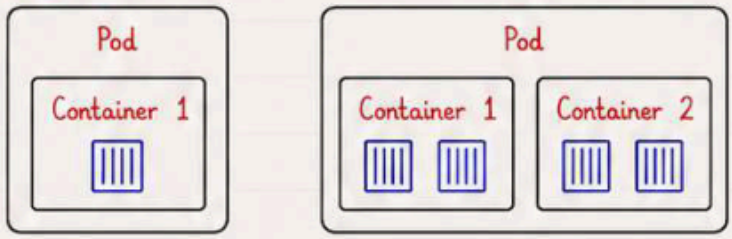
1. What is a Pod ?

A Pod is the smallest deployable unit in Kubernetes.
It represents one or more containers that share the same network, storage and configuration.
Containers in a Pod are always scheduled together on the same node.



2. Pod Types

- **Single Container Pod**
  Contains only one container.
- **Multi Container Pod**
  Contains multiple containers that work closely together.



3. Pod Lifecycle

A Pod goes through different phases during its lifecycle.



4. Why Pod instead of Container ?

- Pods are the basic building block in K8s.
- Allows multiple containers to work together.
- Shares network, storage and config.
- Provides better resource management.
- Easier scaling and self-healing.

Container (Docker)	Pod (Kubernetes)
Single container	One or more containers
Manual management	Managed by Kubernetes
No self-healing	Self-healing (restarts on failure)
Scaling is manual	Auto scaling is possible
No shared resources	Shares network & storage

In short : Pod is the smallest, simplest unit in Kubernetes that Kubernetes creates or deploys. It wraps one or more containers that are tightly coupled.

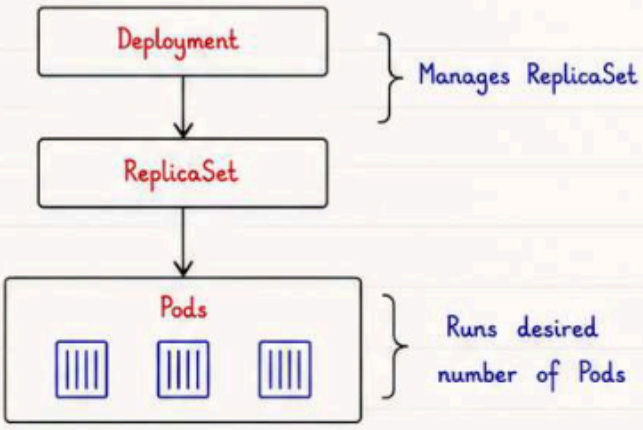
KUBERNETES NOTES

1. Deployments

A Deployment manages ReplicaSets and provides declarative updates for Pods and ReplicaSets.

It helps in :

- Scaling
- Rolling Updates
- Rollback
- Self-healing



2. ReplicaSet

ReplicaSet ensures that a specified number of Pod replicas are running at any time. If a Pod fails, ReplicaSet creates a new one to maintain the desired state.

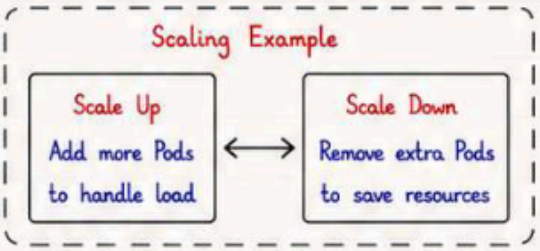
Key Points

- Ensures high availability
- Maintains the desired number of Pods
- Monitors Pods continuously
- Works under Deployment

3. Scaling

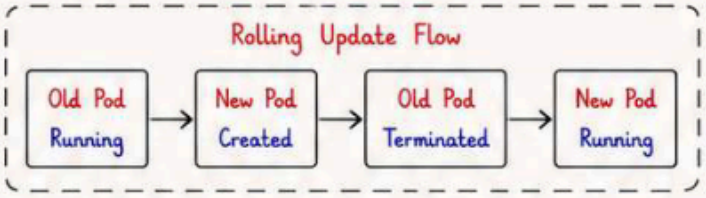
We can scale the number of Pods up or down as per the load.

- Manual Scaling : Using kubectl scale
- Auto Scaling : Using HPA (Horizontal Pod Autoscaler)



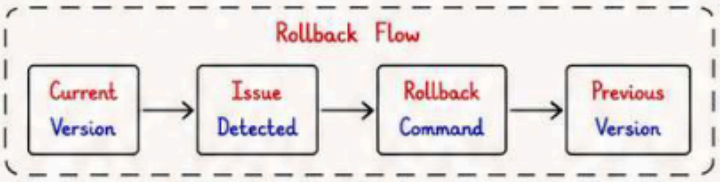
4. Rolling Update

Rolling Update allows us to update the application without downtime.
New Pods are created one by one and old Pods are terminated gradually.



5. Rollback

If something goes wrong after an update, we can rollback to the previous stable version easily.



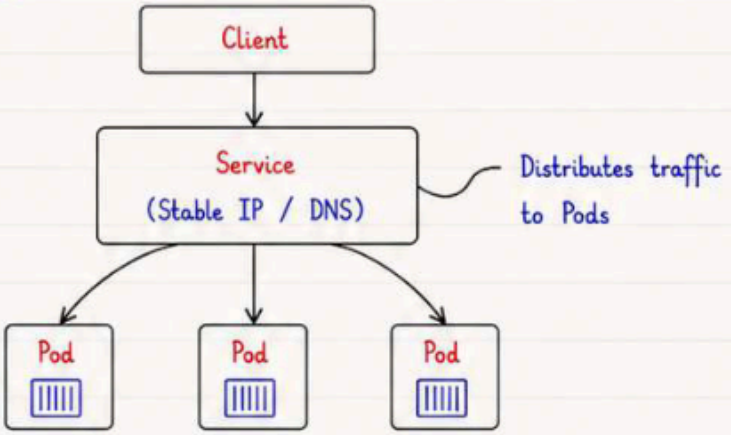
6. kubectl Commands (Deployments)

Command	Description	Example
kubectl get deployments	List all deployments	kubectl get deployments
kubectl create deployment <name> --image=<image>	Create a new deployment	kubectl create deployment myapp --image=nginx
kubectl scale deployment <name> --replicas=<n>	Scale deployment to n replicas	kubectl scale deployment myapp --replicas=5
kubectl rollout status deployment/<name>	Check rollout status	kubectl rollout status deployment/myapp
kubectl rollout undo deployment/<name>	Rollback to previous version	kubectl rollout undo deployment/myapp
kubectl set image deployment/<name> <container>=<image>	Update image of container	kubectl set image deployment/myapp nginx=nginx:1.25

KUBERNETES NOTES

1. Services - Why ?

Pods are ephemeral (can be created, destroyed, recreated).
Their IPs keep changing.
Service provides a stable IP and DNS name to access Pods.
It also helps in load balancing.

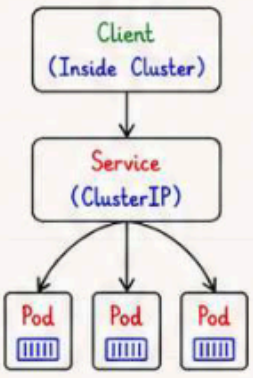


2. Service Types

Type	Use	Accessible From	Description
ClusterIP	Internal communication inside cluster	Inside the cluster	Default type. Exposes the Service on a cluster-internal IP.
NodePort	Expose service on each Node's IP	Outside the cluster (Via Node IP : Port)	Exposes the Service on a static port on each Node's IP.
LoadBalancer	Expose service to external traffic	Outside the cluster (Cloud Load Balancer)	Uses cloud provider's load balancer to expose the Service.
ExternalName	Map service to an external domain	Inside the cluster	Maps the Service to a DNS name outside the cluster.

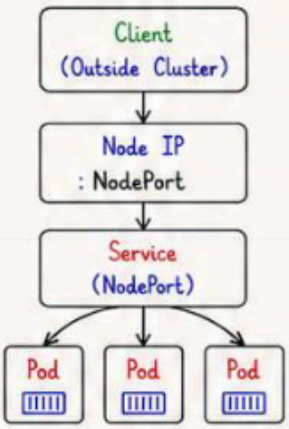
3. Service Types - Diagrams

ClusterIP



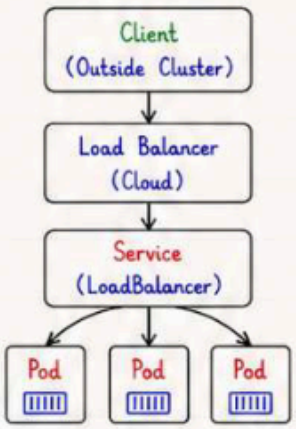
Accessible only inside the cluster.

NodePort



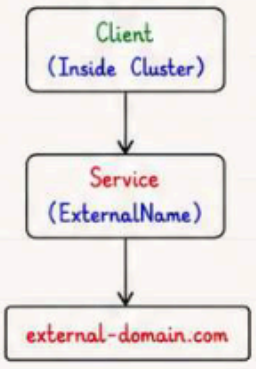
Accessible from outside via Node IP and NodePort.

LoadBalancer



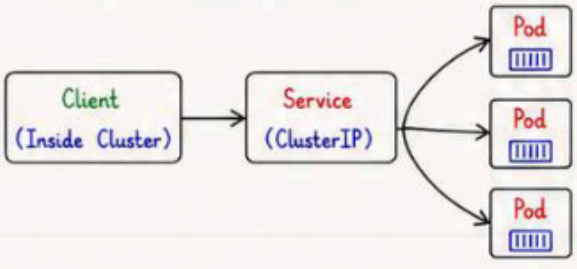
Accessible from outside via Cloud Load Balancer.

ExternalName



Maps to external DNS name.

4. Service Example (ClusterIP)



Example

- Service Name : my-service
- Type : ClusterIP
- Cluster IP : 10.96.0.15
- Port : 80
- Target Port : 8080

Client inside cluster accesses my-service:80

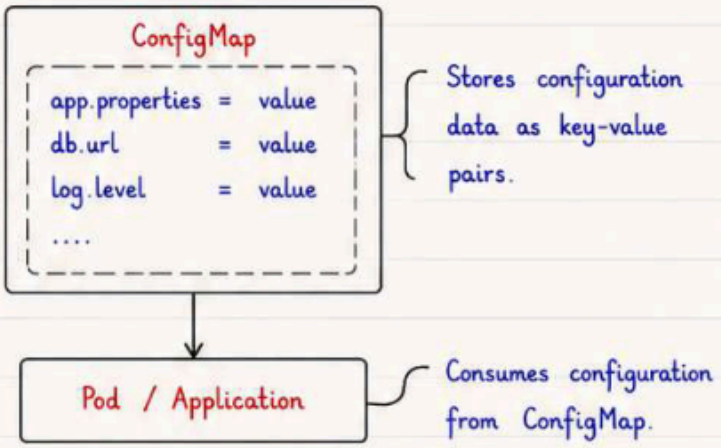
- Traffic is load balanced across Pods.

KUBERNETES NOTES

— Page 6 —

1. ConfigMap - Why ?

ConfigMap is used to store non-sensitive configuration data in key-value pairs.
It allows us to separate configuration from container image.



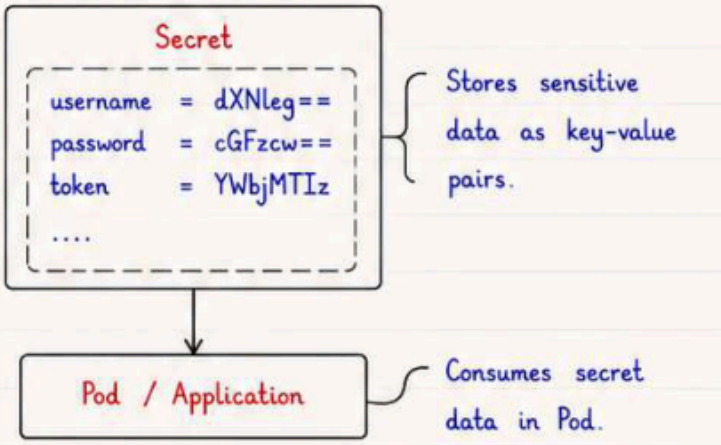
2. ConfigMap - Use Cases

- Application configuration
- Environment variables
- Command line arguments
- Configuration files

---

3. Secret - Why ?

Secret is used to store sensitive data like passwords, tokens, keys, etc.
It is Base64 encoded (not encrypted) and more secure than ConfigMap.



4. Secret - Use Cases

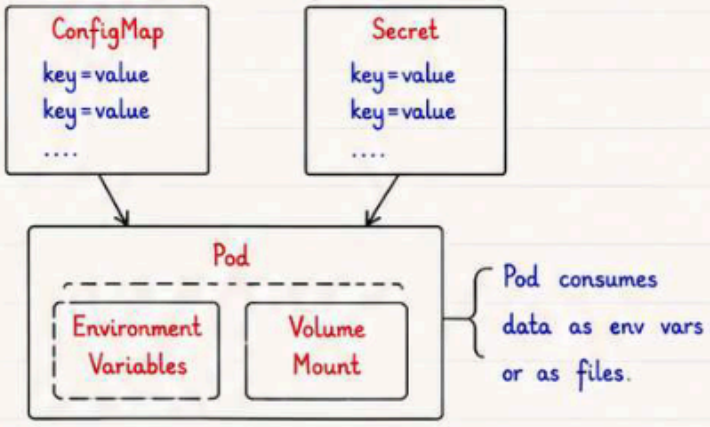
- Passwords
- API Keys
- OAuth Tokens
- TLS Certificates

---

5. ConfigMap vs Secret

Feature	ConfigMap	Secret
Data Type	Non-sensitive	Sensitive
Encoding	Plain Text	Base64 Encoded
Purpose	Configuration	Secrets (passwords, keys)
Security	Less Secure	More Secure
Example	app.properties	db-password

6. How ConfigMap & Secret are used in Pod



7. Important Points

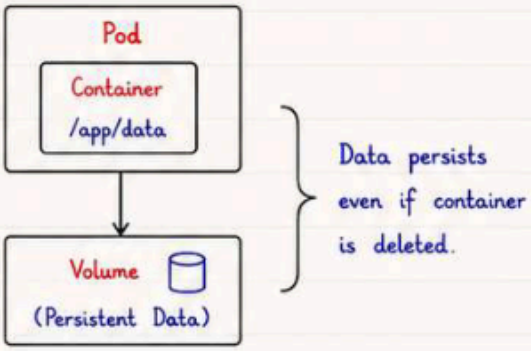
- ConfigMap and Secret help in decoupling configuration from application.
- Secrets are not encrypted by default in etcd. Enable encryption for better security.
- Both can be used as Environment Variables or mounted as Volume inside Pods.
- Avoid storing sensitive data in ConfigMap.

KUBERNETES NOTES

— Page 7 —

1. Volumes - Why ?

Containers are ephemeral.
When a container is deleted,
all data inside it is lost.
Volumes provide persistent storage
that survives container restarts
and deletions.



2. Volume Types

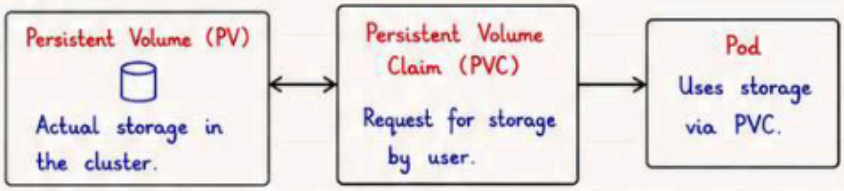
- emptyDir : Temporary storage, deleted when Pod is removed.
- hostPath : Mounts a file or directory from the host node.
- nfs : Mounts NFS share.
- configMap : Mounts ConfigMap as files.
- secret : Mounts Secret as files.
- persistentVolumeClaim : Uses persistent storage (PV).

Notes

Use emptyDir for temporary data.
Use hostPath with caution.
Use PVC for production
workloads.

3. Persistent Storage - PV and PVC

- **PV (Persistent Volume)**
  It is a piece of storage in the cluster provisioned by admin or dynamically.
- **PVC (Persistent Volume Claim)**
  It is a request for storage by users. It binds to a suitable PV.



Flow :
Pod requests storage → PVC → Binds to PV → Pod uses it

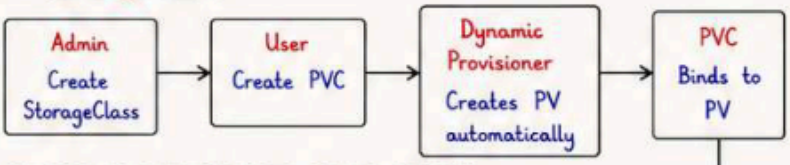
4. Storage Class

StorageClass provides a way to describe the "class" of storage you want.
It enables dynamic provisioning of PVs.

Example

fast-ssd	:	For high performance
standard	:	For general purpose
slow-hdd	:	For backup, logs

5. Storage Flow



Notes

If no StorageClass is defined, static PV needs to be created manually.
Dynamic provisioning is recommended.

6. Useful Commands (Volumes)

Command	Description	Example
kubectl get pv	List all Persistent Volumes	kubectl get pv
kubectl get pvc	List all Persistent Volume Claims	kubectl get pvc
kubectl describe pvc <name>	Describe a PVC	kubectl describe pvc my-pvc
kubectl delete pvc <name>	Delete a PVC	kubectl delete pvc my-pvc
kubectl get storageclass	List all Storage Classes	kubectl get storageclass
kubectl describe storageclass <name>	Describe a Storage Class	kubectl describe storageclass fast-ssd

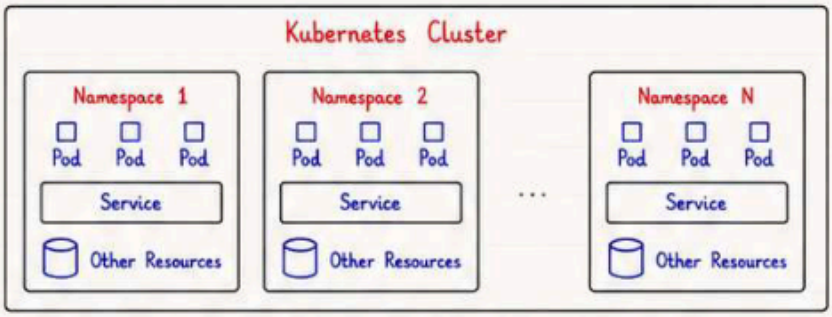
KUBERNETES NOTES

— Page 8 —

1. Namespaces

Namespace provides a way to divide cluster resources between multiple users, teams or environments.

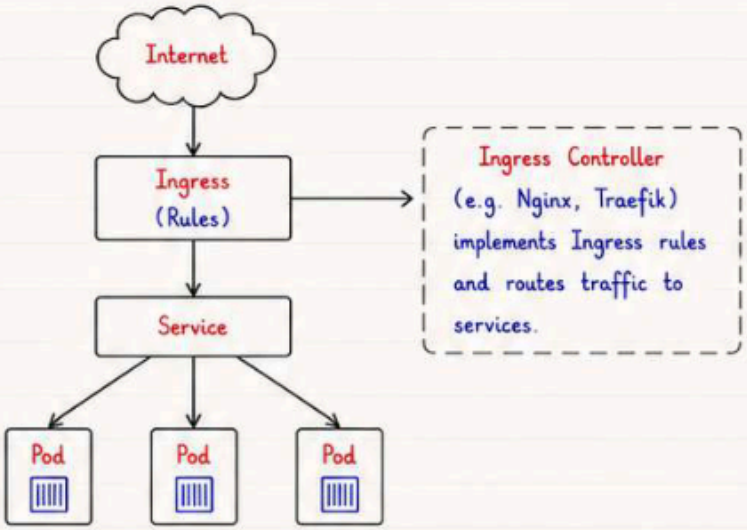
- Default namespace is "default"
- Helps in isolation and resource management.
- Resources in one namespace are not visible in another.



> **Note :**
> Resources in different namespaces are isolated from each other.
> A pod in namespace 1 cannot access a pod in namespace 2 directly.

2. Ingress - Why ?

Service of type LoadBalancer/NodePort exposes applications, but Ingress provides more powerful routing (host based / path based) using a single entry point.



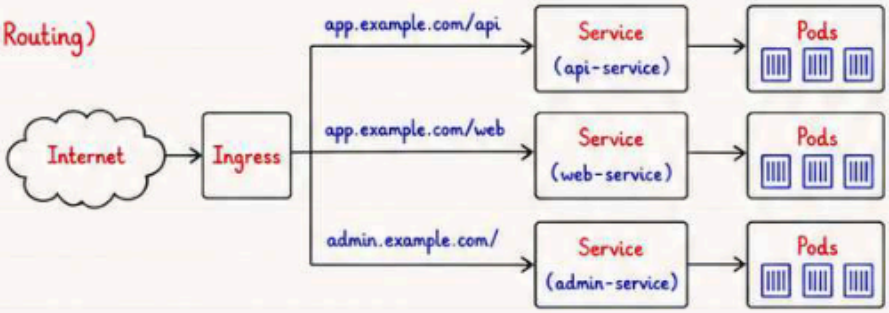
3. Ingress - Key Points

- Works at Layer 7 (Application Layer).
- Supports host based routing.
- Supports path based routing.
- TLS/SSL termination.
- Works with Ingress Controller.

4. Ingress Example (Host & Path based Routing)

Example Rules

Host : app.example.com
  Path : /api → Service : api-service
  Path : /web → Service : web-service
Host : admin.example.com
  Path : / → Service : admin-service



5. Service vs Ingress

Feature	Service	Ingress
Layer	Layer 4 (TCP/UDP)	Layer 7 (HTTP/HTTPS)
Routing	Basic (Port based)	Advanced (Host / Path based)
SSL/TLS	Not handled	Can handle SSL termination
Usage	Expose service	Expose multiple services via rules
Controller	Not required	Requires Ingress Controller

6. Useful Commands

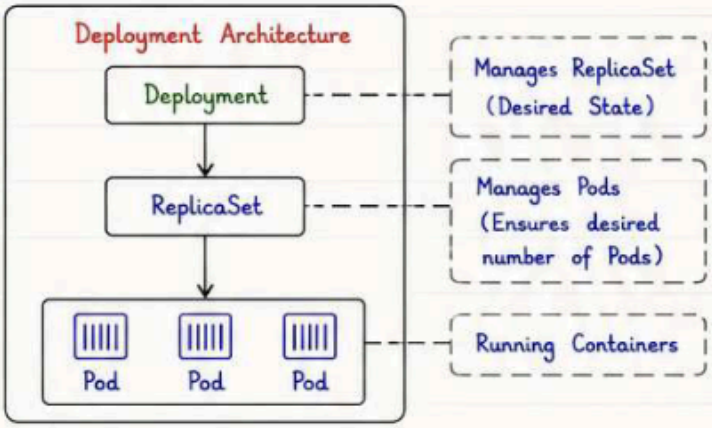
Command	Description
kubectl get ns	List all namespaces
kubectl get ingress -A	List all Ingress in all namespaces
kubectl describe ingress <name>	Describe Ingress
kubectl apply -f ingress.yaml	Create Ingress from YAML
kubectl delete ingress <name>	Delete Ingress

In short : Namespaces help in isolation. Ingress helps in smart routing of traffic to services using host and path rules.

KUBERNETES NOTES

1. Deployments - Why ?

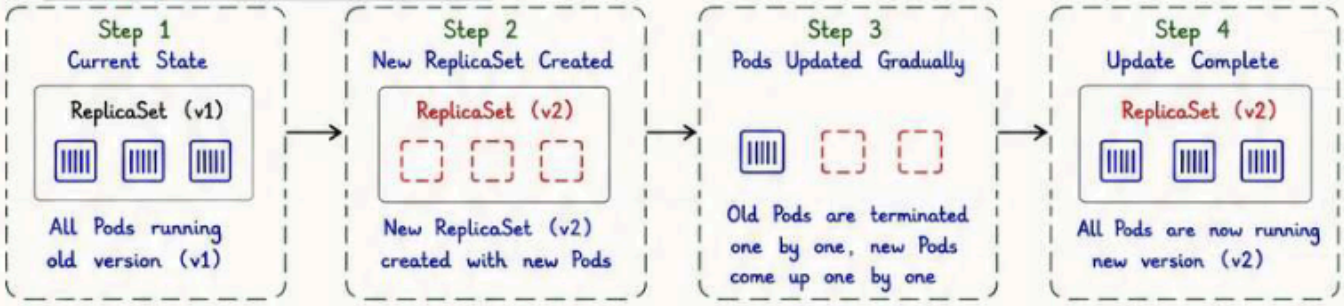
- Deployments provide declarative updates for Pods and ReplicaSets.
- They help in managing application lifecycle.
- Supports rolling updates and rollbacks.
- Self-healing : ensures desired number of Pods are running.
- Acts as a higher-level abstraction over ReplicaSet.



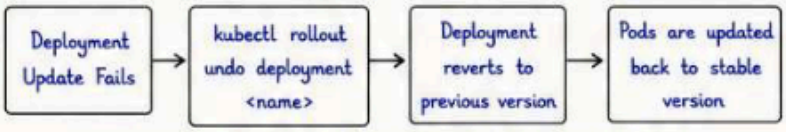
2. ReplicaSet vs Deployment

Feature	ReplicaSet	Deployment
Purpose	Ensures a specified number of Pod replicas are running.	Manages ReplicaSets and provides declarative updates.
Updates	No rolling updates.	Supports rolling updates.
Rollbacks	Not supported.	Supports rollbacks to previous versions.
Scaling	Supported (Replicas).	Supported (Replicas).
Self-healing	Yes, maintains desired state.	Yes, maintains desired state.
Use Case	Use directly in simple use cases.	Use in production for managing applications.

3. Rolling Update - How it Works ?



4. Rollback - How it Works ?



Example Command

```
kubectl rollout undo deployment my-app
kubectl rollout history deployment my-app
```

5. Deployment Strategies

- Recreate (Default for some resources)
  - All old Pods are terminated first.
  - New Pods are created after.
- RollingUpdate (Default for Deployments)
  - Pods are updated one by one.
  - Zero downtime.
- Blue/Green Deployment (Advanced)
  - Two environments (Blue & Green).
  - Switch traffic to new version.

6. Useful Commands (Deployments)

Command	Description	Example
kubectl create deployment <name> --image=<image>	Create a new Deployment	kubectl create deployment myapp --image=nginx
kubectl get deployments	List all Deployments	kubectl get deployments
kubectl get rs	List all ReplicaSets	kubectl get rs
kubectl get pods	List all Pods	kubectl get pods
kubectl describe deployment <name>	Describe a Deployment	kubectl describe deployment myapp
kubectl scale deployment <name> --replicas=<n>	Scale Deployment	kubectl scale deployment myapp --replicas=5
kubectl rollout status deployment <name>	Check rollout status	kubectl rollout status deployment myapp
kubectl rollout history deployment <name>	Show rollout history	kubectl rollout history deployment myapp
kubectl rollout undo deployment <name>	Rollback to previous version	kubectl rollout undo deployment myapp

KUBERNETES NOTES

— Page 10 —

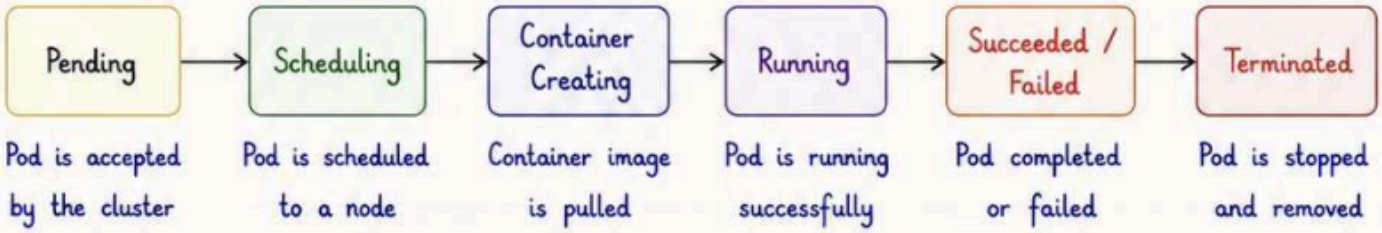
1. ConfigMaps vs Secrets (Quick View)

ConfigMap	Secret
• Stores non-sensitive configuration data • Plain text (Base64 not used) • Used as environment variables or config files • Use for app settings, URLs, feature flags	• Stores sensitive data • Base64 encoded (not encrypted) • Used as environment variables or mounted as files • Use for passwords, tokens, keys, certificates

2. Resource Requests vs Limits

Requests	• Minimum resources required • Used by scheduler to place Pods • If not set, may lead to poor scheduling
Limits	• Maximum resources a container can use • Prevents one container from using all resources • If exceeded, container may be throttled or killed

3. Pod Lifecycle (In Short)



4. Node Types

- Master Node (Control Plane):
  Manages the cluster, makes global decisions.
- Worker Node:
  Runs the application Pods.
- etcd Node:
  Stores all cluster data (key-value store).

5. Common Labels

- app : Name of the application
- env : Environment (dev, test, prod)
- tier : Tier (frontend, backend)
- version : Version of the application
- team : Owning team

6. Health Checks (Probes)

- Liveness Probe : Is the container running?
- Readiness Probe : Is the container ready to serve traffic?
- Startup Probe : Is the application started?

7. Useful kubectl Commands

- kubectl get pods → List all Pods
- kubectl get svc → List all Services
- kubectl get nodes → List all Nodes
- kubectl describe pod <pod-name> → Details of a Pod
- kubectl logs <pod-name> → Logs of a Pod
- kubectl exec -it <pod-name> -- /bin/bash → Shell into Pod